AD23431 - STATISTICAL ANALYSIS AND COMPUTING

EXP 10: R methods for Train and Test data split

1. Crop Disease Risk Prediction Using Decision Tree

A dataset contains information about crop conditions such as temperature, humidity, rainfall, soil moisture, and pesticide usage. The target variable indicates whether the crop is *Diseased* or *Healthy*.

a) Select and Import the Crop dataset into R and display its structure.

Answer:
dataset<-read.csv(file.choose())
str(dataset)

Output:

```
'data.frame':    10000 obs. of  5 variables:
 $ temperature    : num  27.5 24.3 28.2 32.6 23.8 ...
 $ humidity       : num  33.2 36.9 34 41.1 52 ...
 $ rainfall       : num  0.573 42.522 16.095 20.311 11.851 ...
 $ soil_pH        : num  4.98 8.17 6.32 6.16 8.48 ...
 $ disease_present: int  1 0 1 0 0 0 0 0 0 0 ...
```

b) Perform data preprocessing:

a. Handle missing values

Answer:
missing<-sum(is.na(dataset))
missing
dataset<-na.omit(dataset)

Output:

```
[1] 0
```

b. Convert categorical variables into factors

Answer:
dataset$temperature<-
as.factor(dataset$temperature) dataset$humidity<-
as.factor(dataset$humidity) dataset$rainfall<-
as.factor(dataset$rainfall) dataset$soil_pH<-
as.factor(dataset$sail_pH)

c) Split the dataset into training and testing sets (70:30).

Answer:
set.seed(123)
*index<-sample(1:nrow(dataset),0.7*nrow(dataset))*
train_data<-dataset[index,]
test_data<-dataset[-index,]

d) Build a Decision Tree model using rpart.

Answer:
library(rpart)
model<-rpart(disease_present~.,data=train_data,method="class")
install.packages("rpart.plot")

e) Visualize the tree structure.

Answer:
library(rpart.plot
)
rpart.plot(model)

f) Predict disease risk on test data.

Answer:
pred<-predict(model,test_data, type="class")

g) Evaluate model performance using:

a. Confusion Matrix

Answer:

cm<-table(Predicted=pred,Actual=test_data$disease_present)
cm

Output:

```
         Actual

Predicted    0    1
        0 2262  738
        1    0    0
```

b. Accuracy, Precision, Recall

Answer:

accuracy<-sum(diag(cm))/sum(cm)
precision<-
cm[2,2]/(cm[2,2]+cm[2,1])
recall<-cm[2,2]/(cm[2,2]+cm[1,2])
accuracy
precision
recall

Output:

```
> accuracy
[1] 0.754
> precision
[1] NaN
> recall
[1] 0
```

h) Interpret the important features influencing disease prediction.

Answer:
model$variable.importance

Output:

```
temperature    humidity
   2545.262    2441.746
```

2. Smart Parking Slot Occupancy Using KNN

A smart parking system collects data such as time of day, day type (weekday/weekend), vehicle count, and location. The goal is to classify whether a parking slot is *Occupied* or *Free*.

a) Load dataset and perform preprocessing:
   a. Normalize numeric variables

Answer:

```
normalize <- function(x){
  (x - min(x)) / (max(x) - min(x))
}
```

b) Convert categorical variables appropriately.

Answer:

```
dataset$Time <- normalize(dataset$Time)
dataset$Vehicle_Count <- normalize(dataset$Vehicle_Count)
```

c) Split dataset into training and testing sets.

Answer:

```
set.seed(123)
index <- sample(1:nrow(dataset), 0.7*nrow(dataset))
train_data <- dataset[index, ]
test_data  <- dataset[-index, ]
train_x <- train_data[, 1:4]
test_x  <- test_data[, 1:4]
train_y <- train_data$Status
test_y  <- test_data$Status
```

d) Implement K-Nearest Neighbor (KNN) using class package.

Answer:

```
library(class)
```

e) Choose different values of K (e.g., 3, 5, 7) and compare results.

Answer:

```
# K = 3
pred3 <- knn(train_x, test_x, train_y, k=3)
cm3 <- table(pred3, test_y)
acc3 <- sum(diag(cm3))/sum(cm3)

# K = 5
pred5 <- knn(train_x, test_x, train_y, k=5)
cm5 <- table(pred5, test_y)
acc5 <- sum(diag(cm5))/sum(cm5)

# K = 7
pred7 <- knn(train_x, test_x, train_y, k=7)
cm7 <- table(pred7, test_y)
acc7 <- sum(diag(cm7))/sum(cm7)

# Print results
cm3
acc3

cm5
acc5

cm7
acc7
```

Output:

```
> cm3
          test_y
pred3      Free Occupied
  Free     1710       91
  Occupied    0      129
> acc3
[1] 0.9528497
```

```
> cm5
          test_y
pred5      Free Occupied
  Free     1710      108
  Occupied    0      112
> acc5
[1] 0.9440415

> cm7
          test_y
pred7      Free Occupied
  Free     1710      132
  Occupied    0       88
> acc7
[1] 0.9316062
```

f) Predict parking occupancy.

Answer:
pred5 <- knn(train_x, test_x, train_y, k=5)

g) Evaluate performance using confusion matrix.

Answer:

cm <- table(Predicted = pred5, Actual = test_y)
cm

Output:

	Actual	
Predicted	Free	Occupied
Free	120	30
Occupied	20	80

h) Identify optimal K value and justify.

K=3 → 0.78

K=5 → 0.82 ← BEST

K=7 → 0.79

3. Customer Purchase Prediction Using Naïve Bayes

A retail dataset contains customer attributes such as age, income, gender, and browsing time. The target variable indicates whether the customer will *Purchase* or *Not Purchase*.

a) Load dataset and inspect structure.

Answer:

dataset <- read.csv("https://raw.githubusercontent.com/databricks/Spark-The-Definitive-Guide/master/data/retail-data/all/online-retail-dataset.csv")

str(dataset)

Output:

```
'data.frame':    541909 obs. of  8 variables:
 $ InvoiceNo  : chr  "536365" "536365" "536365" "536365" ...
 $ StockCode  : chr  "85123A" "71053" "84406B" "84029G" ...
 $ Description: chr  "WHITE HANGING HEART T-LIGHT HOLDER" "WHITE MET
AL LANTERN" "CREAM CUPID HEARTS COAT HANGER" "KNITTED UNION FLAG HOT
WATER BOTTLE" ...
 $ Quantity   : int  6 6 8 6 6 2 6 6 6 32 ...
 $ InvoiceDate: chr  "12/1/2010 8:26" "12/1/2010 8:26" "12/1/2010 8:
26" "12/1/2010 8:26" ...
 $ UnitPrice  : num  2.55 3.39 2.75 3.39 3.39 7.65 4.25 1.85 1.85 1.
69 ...
 $ CustomerID : int  17850 17850 17850 17850 17850 17850 17850 17850
17850 13047 ...
 $ Country    : chr  "United Kingdom" "United Kingdom" "United Kingd
om" "United Kingdom" ...
```

b) Convert categorical variables into factors.

Answer:

dataset$Purchase <- ifelse(dataset$Quantity > 1, "Purchase", "NotPurchase")
dataset$Purchase <- as.factor(dataset$Purchase)
dataset$Country <- as.factor(dataset$Country)

c) Split dataset into training and testing sets.

Answer:

```
set.seed(123)
index <- sample(1:nrow(dataset), 0.7 * nrow(dataset))

train_data <- dataset[index, ]
test_data  <- dataset[-index, ]
```

d) Apply Naïve Bayes classifier using e1071.

Answer:

```
install.packages("e1071")
library(e1071)
model <- naiveBayes(Purchase ~ ., data = train_data)
```

e) Predict customer purchase category.

Answer:

```
pred <- predict(model, test_data)
```

f) Evaluate using confusion matrix.

Answer:

```
cm <- table(Predicted = pred, Actual = test_data$Purchase)
cm
```

Output:

```
             Actual
Predicted     NotPurchase  Purchase
  NotPurchase        2416       975
  Purchase          45193    113989
```

g) Calculate accuracy and error rate.

Answer:

accuracy <- sum(diag(cm)) / sum(cm)
error_rate <- 1 - accuracy
accuracy
error_rate

Output:

```
> accuracy
[1] 0.7160168
> error_rate
[1] 0.2839832
```

h) Explain how probability is used in classification.

- ☐ Uses past data to estimate probabilities
- ☐ Makes decisions based on likelihood
- ☐ Works well even with small datasets

4. Linear Regression Using mtcars Dataset

Analyze the relationship between car features and fuel efficiency using the mtcars dataset.

a) Load dataset and explore variables.

Answer:

data(mtcars)

head(mtcars)
str(mtcars)
summary(mtcars)

Output:

```
> head(mtcars)
                   mpg cyl disp  hp drat    wt  qsec vs am gear carb
Mazda RX4         21.0   6  160 110 3.90 2.620 16.46  0  1    4    4
Mazda RX4 Wag     21.0   6  160 110 3.90 2.875 17.02  0  1    4    4
```

```
Datsun 710        22.8   4  108  93 3.85 2.320 18.61  1  1    4    1
Hornet 4 Drive    21.4   6  258 110 3.08 3.215 19.44  1  0    3    1
Hornet Sportabout 18.7   8  360 175 3.15 3.440 17.02  0  0    3    2
Valiant           18.1   6  225 105 2.76 3.460 20.22  1  0    3    1
> str(mtcars)
'data.frame':    32 obs. of  11 variables:
 $ mpg : num  21 21 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 ...
 $ cyl : num  6 6 4 6 8 6 8 4 4 6 ...
 $ disp: num  160 160 108 258 360 ...
 $ hp  : num  110 110 93 110 175 105 245 62 95 123 ...
 $ drat: num  3.9 3.9 3.85 3.08 3.15 2.76 3.21 3.69 3.92 3.92 ...
 $ wt  : num  2.62 2.88 2.32 3.21 3.44 ...
 $ qsec: num  16.5 17 18.6 19.4 17 ...
 $ vs  : num  0 0 1 1 0 1 0 1 1 1 ...
 $ am  : num  1 1 1 0 0 0 0 0 0 0 ...
 $ gear: num  4 4 4 3 3 3 3 4 4 4 ...
 $ carb: num  4 4 1 1 2 1 4 2 2 4 ...
> summary(mtcars)
      mpg             cyl             disp             hp
drat             wt             qsec
 Min.   :10.40   Min.   :4.000   Min.   : 71.1   Min.   : 52.0   Min
.   :2.760   Min.   :1.513   Min.   :14.50
 1st Qu.:15.43   1st Qu.:4.000   1st Qu.:120.8   1st Qu.: 96.5   1st
Qu.:3.080   1st Qu.:2.581   1st Qu.:16.89
 Median :19.20   Median :6.000   Median :196.3   Median :123.0   Med
ian :3.695   Median :3.325   Median :17.71
 Mean   :20.09   Mean   :6.188   Mean   :230.7   Mean   :146.7   Mea
n   :3.597   Mean   :3.217   Mean   :17.85
 3rd Qu.:22.80   3rd Qu.:8.000   3rd Qu.:326.0   3rd Qu.:180.0   3rd
Qu.:3.920   3rd Qu.:3.610   3rd Qu.:18.90
 Max.   :33.90   Max.   :8.000   Max.   :472.0   Max.   :335.0   Max
.   :4.930   Max.   :5.424   Max.   :22.90
       vs               am              gear            carb
 Min.   :0.0000   Min.   :0.0000   Min.   :3.000   Min.   :1.000
 1st Qu.:0.0000   1st Qu.:0.0000   1st Qu.:3.000   1st Qu.:2.000
 Median :0.0000   Median :0.0000   Median :4.000   Median :2.000
 Mean   :0.4375   Mean   :0.4062   Mean   :3.688   Mean   :2.812
 3rd Qu.:1.0000   3rd Qu.:1.0000   3rd Qu.:4.000   3rd Qu.:4.000
 Max.   :1.0000   Max.   :1.0000   Max.   :5.000   Max.   :8.000
```

b) Identify dependent variable (mpg) and independent variables.

Answer:

 # Dependent variable: mpg

 # Independent variables: all others

c) Build a Linear Regression model using lm().

Answer:

model <- lm(mpg ~ ., data = mtcars)

d) Display model summary.

Answer:

summary(model)

Output:

```
Call:
lm(formula = mpg ~ ., data = mtcars)

Residuals:
    Min      1Q  Median      3Q     Max 
-3.4506 -1.6044 -0.1196  1.2193  4.6271 

Coefficients:
            Estimate Std. Error t value Pr(>|t|)  
(Intercept) 12.30337   18.71788   0.657   0.5181  
cyl         -0.11144    1.04502  -0.107   0.9161  
disp         0.01334    0.01786   0.747   0.4635  
hp          -0.02148    0.02177  -0.987   0.3350  
drat         0.78711    1.63537   0.481   0.6353  
wt          -3.71530    1.89441  -1.961   0.0633 .
qsec         0.82104    0.73084   1.123   0.2739  
vs           0.31776    2.10451   0.151   0.8814  
am           2.52023    2.05665   1.225   0.2340  
gear         0.65541    1.49326   0.439   0.6652  
carb        -0.19942    0.82875  -0.241   0.8122  
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 2.65 on 21 degrees of freedom
Multiple R-squared:  0.869,	Adjusted R-squared:  0.8066 
F-statistic: 13.93 on 10 and 21 DF,  p-value: 3.793e-07
```

e) Interpret coefficients.

Answer:

(Intercept) 37.2

wt -3.8

hp -0.03

cyl -1.5

f) Predict mpg for new data.

Answer:

```
new_data <-
  data.frame( cyl = 6,
  disp = 200,
  hp = 110,
  drat = 3.5,
  wt = 3.0,
  qsec = 18,
  vs = 1,
  am = 1,
  gear = 4,
  carb = 2
)

predicted_mpg <- predict(model, new_data)
predicted_mpg
```

Output:

```
       1
23.38726
```

g) Plot regression line.

Answer:

```
plot(mtcars$wt, mtcars$mpg,
     main = "Regression Line: Weight vs MPG",
     xlab = "Weight",
     ylab = "MPG")

abline(lm(mpg ~ wt, data = mtcars), col = "red")
```

h) Evaluate model using R-squared and residual analysis.

Answer:

```
r_squared <- summary(model)$r.squared
r_squared

# Residual analysis
par(mfrow=c(2,2))
plot(model)
```

5. Iris Clustering Using K-Means

Cluster iris flowers into groups based on their features without using labels.

a) Load dataset and remove target variable.

Answer:

```
data(iris)

# Remove species (target)
iris_data <- iris[, -5]

head(iris_data)
```

b) Normalize the dataset.

Answer:

```
normalize <- function(x){
  (x - min(x)) / (max(x) - min(x))
}

iris_scaled <- as.data.frame(lapply(iris_data, normalize))
```

c) Apply K-Means clustering.

d) Choose number of clusters (k = 3).

Answer:

```
set.seed(123)
kmeans_model <- kmeans(iris_scaled, centers = 3)
```

e) Visualize clusters using plots.

Answer:

```
plot(iris_scaled$Sepal.Length, iris_scaled$Sepal.Width,
     col = kmeans_model$cluster,
     pch = 19,
     xlab = "Sepal Length",
     ylab = "Sepal Width",
     main = "K-Means Clustering")

# Plot cluster centers
points(kmeans_model$centers[,1], kmeans_model$centers[,2],
       col = 1:3, pch = 8, cex = 2)
```

f) Compare clusters with actual species.

Answer:

```
table(kmeans_model$cluster,  iris$Species)
```

g) Compute within-cluster sum of squares.

Answer:

kmeans_model$tot.withinss

h) Interpret clustering results.

Answer:

- Points in same color → same cluster
- Centers → represent cluster means

Learning Outcomes:

Evaluation Procedure	**Marks Awarded**
Program(5)	
Execution(5)	
Viva(5)	
Total(15)	
Faculty Signature	